

Untitled

Jack

2026-09-21

#Setup code

```
student_id <- c("S01", "S02", "S03", "S04", "S05", "S06")
section <- c("A", "B", "A", "B", "A", "B")
quiz1 <- c(82, 91, 76, 88, 95, 69)
quiz2 <- c(85, 89, 80, 92, 94, 74)
passed <- c(TRUE, TRUE, TRUE, TRUE, TRUE, FALSE)
```

Part A

```
section <- factor(section, levels = c("A", "B"))

# data frame containing all five vectors
students <- data.frame(
  student_id = student_id,
  section     = section,
  quiz1       = quiz1,
  quiz2       = quiz2,
  passed      = passed,
  stringsAsFactors = FALSE
)

# matrix: students as rows, quizzes as columns
score_matrix <- matrix(
  c(quiz1, quiz2),
  nrow = 6, ncol = 2,
  dimnames = list(student_id, c("quiz1", "quiz2"))
)

# list containing course, scores, and a named cutoffs vector
course_record <- list(
  course = "R Programming",
  scores = students,
  cutoffs = c(pass = 70, excellent = 90)
)

# Inspect the objects
typeof(section);      class(section);      length(section);      str(section)

## [1] "integer"
## [1] "factor"
```

```

## [1] 6
## Factor w/ 2 levels "A","B": 1 2 1 2 1 2
typeof(students);      class(students);      length(students);      str(students);      dim(students)

## [1] "list"
## [1] "data.frame"
## [1] 5
## 'data.frame':      6 obs. of  5 variables:
## $ student_id: chr  "S01" "S02" "S03" "S04" ...
## $ section   : Factor w/ 2 levels "A","B": 1 2 1 2 1 2
## $ quiz1     : num  82 91 76 88 95 69
## $ quiz2     : num  85 89 80 92 94 74
## $ passed    : logi  TRUE TRUE TRUE TRUE TRUE FALSE
## [1] 6 5
typeof(score_matrix); class(score_matrix); length(score_matrix); str(score_matrix); dim(score_matrix)

## [1] "double"
## [1] "matrix" "array"
## [1] 12
## num [1:6, 1:2] 82 91 76 88 95 69 85 89 80 92 ...
## - attr(*, "dimnames")=List of 2
## ..$ : chr [1:6] "S01" "S02" "S03" "S04" ...
## ..$ : chr [1:2] "quiz1" "quiz2"
## [1] 6 2
typeof(course_record);class(course_record);length(course_record);str(course_record)

## [1] "list"
## [1] "list"
## [1] 3
## List of 3
## $ course : chr "R Programming"
## $ scores : 'data.frame': 6 obs. of  5 variables:
## ..$ student_id: chr [1:6] "S01" "S02" "S03" "S04" ...
## ..$ section   : Factor w/ 2 levels "A","B": 1 2 1 2 1 2
## ..$ quiz1     : num [1:6] 82 91 76 88 95 69
## ..$ quiz2     : num [1:6] 85 89 80 92 94 74
## ..$ passed    : logi [1:6] TRUE TRUE TRUE TRUE TRUE FALSE
## $ cutoffs: Named num [1:2] 70 90
## ..- attr(*, "names")= chr [1:2] "pass" "excellent"
section <- factor(section, levels = c("A","B"))

students <- data.frame(
  student_id = student_id,
  section = section,
  quiz1 = quiz1,
  quiz2 = quiz2,
  passed = passed,

```

```

    stringAsFactors = FALSE
)

score_matrix <- matrix(
  c(quiz1, quiz2),
  nrow = 6, ncol = 2,
  dimnames = list(student_id, c("quiz1", "quiz2"))
)

course_record <- list(course = "R Programming",
                      score = students,
                      cutoffs = c(pass = 70, excellent = 90))

typeof(quiz1); class(quiz1); length(quiz1)

## [1] "double"
## [1] "numeric"
## [1] 6

typeof(section); class(section); length(section)

## [1] "integer"
## [1] "factor"
## [1] 6

typeof(score_matrix); class(score_matrix); dim(score_matrix)

## [1] "double"
## [1] "matrix" "array"
## [1] 6 2

typeof(students); class(students); dim(students)

## [1] "list"
## [1] "data.frame"
## [1] 6 6

typeof(course_record); class(course_record); length(course_record)

## [1] "list"
## [1] "list"
## [1] 3

# A vector contains elements of one atomic type
# List: Contain elements of different types and lengths
# Matrix: An atomic vectpr with a 2d dim attribute
# Data Frame: A list of equal-length vector

```

Part B

```
score_matrix["S04", "quiz2"]

## [1] 92
score_matrix[1:2, , drop = FALSE]

##      quiz1 quiz2
## S01      82    85
## S02      91    89
course_record["course"]

## $course
## [1] "R Programming"
course_record[["course"]]

## [1] "R Programming"
course_record$course

## [1] "R Programming"
# [ always preserves the structure. [[ extracts a single
# element and returns the element itself.
# $ returns a named element

# row-wise mean of the two quizzes
students$average <- rowMeans(students[, c("quiz1", "quiz2")])

# TRUE when average >= 90
students$excellent <- students$average >= 90

# section A students with average >= 80
section_A_high <- students[students$section == "A" & students$average >= 80, ]

# only student_id, section, average for that subset
section_A_high[, c("student_id", "section", "average")]

##      student_id section average
## 1          S01      A    83.5
## 5          S05      A    94.5
# named numeric vector of all student averages
avg_by_student <- students$average
names(avg_by_student) <- students$student_id
avg_by_student

##  S01  S02  S03  S04  S05  S06
## 83.5 90.0 78.0 90.0 94.5 71.5
```

Part C

```
students$average <- rowMeans(students[, c("quiz1", "quiz2")])

students$excellent <- students$average >= 90
```

```

section_A <- students[students$section == "A" & students$average >= 80, ]

section_A[, c("student_id", "section", "average")]

##   student_id section average
## 1         S01      A    83.5
## 5         S05      A    94.5

avg_by_student <- students$average
names(avg_by_student) <- students$student_id

# The calculations are vectorized because it is faster than element-by-element loops.

```

Problem 2

```

csv_text <- "sample_id,site,temp_c,ph,status
M01,North,18.2,7.1,ok
M02,South,20.5,,ok
M03,North,NA,6.8,review
M04,East,22.1,7.4,ok
M05,South,19.7,7.0,review
M06,East,23.0,NA,ok
M07,North,17.8,6.9,ok
M08,South,21.2,7.2,ok"

measurements <- read.csv(text = csv_text, na.strings = c("", "NA"),
                          stringsAsFactors = FALSE)

head(measurements)

##   sample_id  site temp_c  ph status
## 1      M01 North   18.2  7.1    ok
## 2      M02 South   20.5  NA    ok
## 3      M03 North    NA  6.8 review
## 4      M04  East   22.1  7.4    ok
## 5      M05 South   19.7  7.0 review
## 6      M06  East   23.0  NA    ok

str(measurements)

## 'data.frame':   8 obs. of  5 variables:
##  $ sample_id: chr  "M01" "M02" "M03" "M04" ...
##  $ site      : chr  "North" "South" "North" "East" ...
##  $ temp_c    : num  18.2 20.5 NA 22.1 19.7 23 17.8 21.2
##  $ ph        : num  7.1 NA 6.8 7.4 7 NA 6.9 7.2
##  $ status    : chr  "ok" "ok" "review" "ok" ...

dim(measurements)

## [1] 8 5

names(measurements)

## [1] "sample_id" "site"      "temp_c"    "ph"        "status"

```

```
colSums(is.na(measurements))

## sample_id      site      temp_c      ph      status
##           0           0           1           2           0

measurements_complete <- measurements[complete.cases(measurements), ]

removed_ids <- measurements$sample_id[!complete.cases(measurements)]

# x == NA because NA is an unknown value so comparisons are unknown
```

Part B

```
measurements$site <- factor(measurements$site)
measurements$status <- factor(measurements$status)
levels(measurements$site)

## [1] "East" "North" "South"
levels(measurements$status)

## [1] "ok" "review"

measurements$temp_f <- measurements$temp_c * 9 / 5 + 32

measurements$ph_below_7 <- measurements$ph < 7

ns_ok <- measurements[
  complete.cases(measurements) &
  measurements$site %in% c("North", "South") &
  measurements$status == "ok",
]

ns_ok[, c("sample_id", "site", "temp_c", "temp_f", "ph")]

## sample_id site temp_c temp_f ph
## 1 M01 North 18.2 64.76 7.1
## 7 M07 North 17.8 64.04 6.9
## 8 M08 South 21.2 70.16 7.2

mean(measurements$temp_c, na.rm = TRUE)

## [1] 20.35714

mean(measurements$temp_c[measurements$site == "South"], na.rm = TRUE)

## [1] 20.46667
```

Part C

```
A <- matrix(1:4, nrow = 2)
B <- matrix(5:8, nrow = 2)

A * B

##      [,1] [,2]
```

```
## [1,]    5   21
## [2,]   12   32
```

```
A %*% B
```

```
##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

```
dim(A * B)
```

```
## [1] 2 2
```

```
dim(A %*% B)
```

```
## [1] 2 2
```

```
#Element-wise require that the matrices have the same dimernsions while matrix multiplication
# does not require these constraints
```

Problem 3

```
#Setup
student_id <- paste0("P", sprintf("%02d", 1:8))
scores <- c(95, 82, NA, 67, 74, 88, 59, 91)
```

```
# Part A
grade_one <- function(
  score,
  a_min = 90,
  b_min = 80,
  c_min = 70,
  d_min = 60
) {
  if (is.na(score)) {
    return(NA_character_)
  } else if (score >= a_min) {
    return("A")
  } else if (score >= b_min) {
    return("B")
  } else if (score >= c_min) {
    return("C")
  } else if (score >= d_min) {
    return("D")
  } else {
    return("F")
  }
}
```

```
grade_one(NA)
```

```
## [1] NA
```

```
grade_one(90)
```

```
## [1] "A"
```

```
grade_one(80)
```

```
## [1] "B"
```

```
grade_one(85)
```

```
## [1] "B"
```

```
grade_one(74)
```

```
## [1] "C"
```

Part B

```
grades <- rep(NA_character_, length(scores))
for (i in seq_along(scores)) {
  grades[i] <- grade_one(scores[i])
}
names(grades) <- student_id
grades
```

```
## P01 P02 P03 P04 P05 P06 P07 P08
```

```
## "A" "B" NA "D" "C" "B" "F" "A"
```

```
# i represents the integer index position.
```

Part C-1

```
summarize_scores <- function(x, na.rm = TRUE, digits = 1) {
  c(
    count = length(x),
    missing = sum(is.na(x)),
    mean = round(mean(x, na.rm = na.rm), digits),
    sd = round(sd(x, na.rm = na.rm), digits),
    min = round(min(x, na.rm = na.rm), digits),
    max = round(max(x, na.rm = na.rm), digits)
  )
}
```

```
summarize_scores(scores)
```

```
##   count missing   mean    sd   min   max
##    8.0      1.0  79.4   13.3  59.0  95.0
```

```
summarize_scores(x = scores, na.rm = TRUE, digits = 2)
```

```
##   count missing   mean    sd   min   max
##    8.00      1.00  79.43  13.28  59.00  95.00
```

```
# Part C-2
```

```
plot_scores <- function(x, ...) {
  plot(seq_along(x), x, ...)
}
```

```
plot_scores(scores, type = "b", pch = 19, xlab = "Position", ylab = "Score", main = "Student Scores")
```

Student Scores

